10-03 — DataSource Management

Section	10 — TypeORM Introduction and Setup
Subtopic	03 — DataSource Management
Estimated Duration	2–2.5 hours
Tags	THEORY · PRACTICAL · TYPESCRIPT
Topics Covered	DataSource initialisation, DataSource options, multiple DataSources, switching connections, connection pooling, testing database connectivity

Learning Objectives — This Subtopic

- Initialise a `DataSource` instance programmatically and understand the full option set available at construction time.
- Distinguish between the `initialize()` call and the `DataSource` constructor, and understand when each runs.
- Configure and manage multiple `DataSource` instances within a single application for read/write splitting or multi-tenancy.
- Apply connection pool settings (`poolSize` / `extra`) and reason about pool sizing for production workloads.
- Write a dedicated connectivity test that validates each `DataSource` before the Express server begins accepting requests.
- Handle `DataSource` lifecycle events (initialise, destroy, reconnect) and integrate them cleanly with Express startup/shutdown.

1. What `DataSource` Actually Represents

In sections 10-01 and 10-02 you saw `DataSource` used to configure TypeORM. This section goes deeper: treating `DataSource` as a first-class runtime object with a lifecycle — something you create, initialise, query through, and eventually destroy.

A `DataSource` is TypeORM's representation of a *database connection or pool of connections*. It is not the connection itself — it is the factory and registry that produces connections on demand, hands them back to the pool when done, and coordinates all the TypeORM subsystems (entity metadata, migrations, subscribers) against a single database target.

> **Analogy — The Database Reception Desk**
> Think of `DataSource` as a hotel reception desk. The desk does not *sleep* guests in rooms itself — it manages a pool of rooms (connections), checks guests in and out, and knows which rooms are occupied. When your code asks for a query, the reception desk assigns a free room (connection), the query executes, and the room is returned to the pool. You do not deal with individual rooms directly; you deal with the desk.

The lifecycle has three distinct phases:

1. **Construction** — `new DataSource(options)`. No network activity. TypeORM reads your configuration and prepares entity metadata.
2. **Initialisation** — `await dataSource.initialize()`. The actual TCP connections to the database are opened and the pool is populated.
3. **Destruction** — `await dataSource.destroy()`. All pooled connections are closed cleanly. Call this during graceful shutdown.

> **Key Insight:** TypeORM versions prior to 0.3.x used a global `createConnection()` function that implicitly stored a default connection. From 0.3.x onwards, `DataSource` is explicit and instance-based — there is no hidden global state. This document assumes TypeORM ≥ 0.3.x.

---

2. Constructing a `DataSource` — The Full Option Set

The `DataSourceOptions` type is a discriminated union — the top-level `type` field selects the driver, and each driver exposes its own additional options. The options below apply broadly to all relational drivers.

2.1 Minimal Viable Configuration (Stage 1)

Start with the smallest valid configuration to understand what is truly required:

`src/data-source.ts`

```
import { DataSource } from 'typeorm';

export const AppDataSource = new DataSource({
  type: 'postgres',
  host: 'localhost',
  port: 5432,
  username: 'postgres',
  password: 'secret',
  database: 'my_app_db',
  entities: [],
  synchronize: false,
});
```

Output:

```
// No output at construction time — this is just configuration.
// Calling initialize() below will produce console output.
```

The `entities` array is empty here because no entities have been defined yet. `synchronize: false` is deliberate — never enable automatic schema synchronisation in production; use migrations (covered in Section 12).

2.2 Loading Entities Dynamically (Stage 2)

Hard-coding every entity class into the `entities` array does not scale. TypeORM accepts glob patterns to load all entity files from a directory:

`src/data-source.ts`

```
import { DataSource } from 'typeorm';
import * as dotenv from 'dotenv';
dotenv.config();

export const AppDataSource = new DataSource({
  type: 'postgres',
  host:     process.env.DB_HOST     ?? 'localhost',
  port:     parseInt(process.env.DB_PORT ?? '5432'),
  username: process.env.DB_USER     ?? 'postgres',
  password: process.env.DB_PASSWORD ?? '',
  database: process.env.DB_NAME     ?? 'my_app_db',

  // Glob: load every .ts file inside src/entities/
  entities: [__dirname + '/entities/**/*.ts'],

  // Load migrations separately — never run them automatically
  migrations: [__dirname + '/migrations/**/*.ts'],

  synchronize: false,
  logging: process.env.NODE_ENV === 'development',
});
```

Output:

```
// Construction is synchronous; no output yet.
// The __dirname glob expands at runtime when initialize() is called.
```

Note — Compiled vs Source Globs: When you compile TypeScript with `tsc`, `.ts` files become `.js` files. The glob pattern must match what actually exists on disk at runtime. If running via `ts-node` or `tsx`, `**/*.ts` is correct. If running the compiled output directly, change to `**/*.js`. A common pattern is:

```
  entities: [__dirname + '/entities/**/*{.ts,.js}'],
```

2.3 The Full Option Reference

The table below documents the most commonly used options across all relational drivers:

Option	Type	Default	Purpose
`type`	`string`	—	Driver name: `'postgres'`, `'mysql'`, `'sqlite'`, etc.
`host`	`string`	`'localhost'`	Database server hostname or IP.
`port`	`number`	Driver-specific	5432 for PostgreSQL, 3306 for MySQL.
`username`	`string`	—	Database login user.
`password`	`string`	—	Database login password.
`database`	`string`	—	Target database (schema) name.
`entities`	`Array`	`[]`	Entity classes or glob patterns.
`migrations`	`Array`	`[]`	Migration classes or glob patterns.
`subscribers`	`Array`	`[]`	Event subscriber classes.
`synchronize`	`boolean`	`false`	Auto-sync schema on init. **Never use in production.**
`logging`	`boolean | string[]`	`false`	`true` logs all SQL; or use `['error', 'warn']`.
`logger`	`string | Logger`	`'advanced-console'`	Custom logging implementation.

Option	Type	Default	Purpose
`poolSize`	`number`	Driver-specific	Max simultaneous connections in the pool.
`extra`	`object`	`{}`	Driver-native options passed directly to the underlying driver.
`ssl`	`object | boolean`	`false`	TLS options for encrypted connections.
`connectTimeoutMS`	`number`	Driver-specific	Timeout in ms before a connection attempt fails.
`migrationsRun`	`boolean`	`false`	Run pending migrations on `initialize()`. Use with caution.
`cache`	`boolean | object`	`false`	Query result caching (covered in Section 12).

3. Initialising the DataSource

`initialize()` is an async method — it returns a `Promise<DataSource>`. Until it resolves, no queries can be made. The standard pattern is to initialise the data source before starting the HTTP server.

3.1 Standalone Initialisation Script (Stage 1)

First, see initialisation in isolation:

src/init-db.ts

```
import { AppDataSource } from './data-source';

async function main(): Promise<void> {
  try {
    await AppDataSource.initialize();
    console.log('Data source initialised successfully.');
    console.log('Driver:', AppDataSource.driver.constructor.name);
    console.log('Is initialised:', AppDataSource.isInitialized);
  } catch (err) {
    console.error('Failed to initialise data source:', err);
    process.exit(1);
  } finally {
    await AppDataSource.destroy();
    console.log('Data source destroyed.');
  }
}

main();
```

Output:

```
Data source initialised successfully.
Driver: PostgresDriver
Is initialised: true
Data source destroyed.
```

`AppDataSource.isInitialized` is a boolean property that reflects whether `initialize()` has completed successfully. It is useful for health checks and guard conditions elsewhere in the application.

3.2 Integrating with Express Startup (Stage 2)

The correct pattern is: initialise the database connection *before* the HTTP server begins listening. This prevents the server from accepting requests before it can handle them:

src/app.ts

```
import express, { Application } from 'express';

const app: Application = express();
app.use(express.json());
```

```
// Routes would be registered here
app.get('/health', (_req, res) => {
  res.json({ status: 'ok' });
});

export default app;
```

src/server.ts

```
import app from './app';
import { AppDataSource } from './data-source';

const PORT = parseInt(process.env.PORT ?? '3000');

async function startServer(): Promise<void> {
  // 1. Initialise database first
  await AppDataSource.initialize();
  console.log('[DB] DataSource initialised.');

  // 2. Only then start accepting HTTP traffic
  app.listen(PORT, () => {
    console.log(`[Server] Listening on http://localhost:${PORT}`);
  });
}

startServer().catch((err) => {
  console.error('[Fatal] Failed to start server:', err);
  process.exit(1);
});
```

Output:

```
[DB] DataSource initialised.
[Server] Listening on http://localhost:3000
```

3.3 Graceful Shutdown (Stage 3)

Failing to close the pool cleanly can leave dangling database connections that count against your server's connection limit. Handle OS signals explicitly:

src/server.ts

```
import app from './app';
import { AppDataSource } from './data-source';
import { Server } from 'http';

const PORT = parseInt(process.env.PORT ?? '3000');

async function startServer(): Promise<void> {
  await AppDataSource.initialize();
  console.log('[DB] DataSource initialised.');

  const server: Server = app.listen(PORT, () => {
    console.log(`[Server] Listening on http://localhost:${PORT}`);
  });

  const shutdown = async (signal: string): Promise<void> => {
    console.log(`\n[Server] ${signal} received — shutting down.`);

    server.close(async () => {
      console.log('[Server] HTTP server closed.');

      await AppDataSource.destroy();
      console.log('[DB] DataSource destroyed.');

      process.exit(0);
    });
  };

  process.on('SIGTERM', () => shutdown('SIGTERM'));
  process.on('SIGINT',  () => shutdown('SIGINT'));
}

startServer().catch((err) => {
  console.error('[Fatal] Failed to start server:', err);
  process.exit(1);
});
```

Output:

```
[DB] DataSource initialised.
[Server] Listening on http://localhost:3000

... (user presses Ctrl+C) ...
```

```
[Server] SIGINT received — shutting down.
[Server] HTTP server closed.
[DB] DataSource destroyed.
```

> **Important:** `server.close()` stops accepting *new* connections but waits for existing requests to complete before calling its callback. Only destroy the `DataSource` inside that callback — not before — otherwise in-flight queries will fail.

---

4. Multiple DataSources

There are several real-world scenarios that require more than one `DataSource`:

- **Read/Write splitting** — writes go to a primary instance; reads go to one or more replicas to reduce primary load.
- **Multi-tenancy** — each tenant has their own database; connections are selected per-request based on the tenant identifier.
- **Heterogeneous databases** — part of your data lives in PostgreSQL, another part in SQLite or MySQL.

4.1 Read/Write Split (Stage 1)

Define two named `DataSource` instances and export each separately:

src/data-source.ts

```
import { DataSource } from 'typeorm';
import * as dotenv from 'dotenv';
dotenv.config();

// Primary — accepts writes
export const WriteDataSource = new DataSource({
  type: 'postgres',
  host:     process.env.DB_PRIMARY_HOST ?? 'localhost',
  port:     parseInt(process.env.DB_PRIMARY_PORT ?? '5432'),
  username: process.env.DB_USER     ?? 'postgres',
  password: process.env.DB_PASSWORD ?? '',
  database: process.env.DB_NAME     ?? 'my_app_db',
```

```
  entities: [__dirname + '/entities/**/*{.ts,.js}'],
  synchronize: false,
  logging: ['error'],
  poolSize: 10,
});

// Replica — reads only
export const ReadDataSource = new DataSource({
  type: 'postgres',
  host:     process.env.DB_REPLICA_HOST ?? 'localhost',
  port:     parseInt(process.env.DB_REPLICA_PORT ?? '5433'),
  username: process.env.DB_USER     ?? 'postgres',
  password: process.env.DB_PASSWORD ?? '',
  database: process.env.DB_NAME     ?? 'my_app_db',
  entities: [__dirname + '/entities/**/*{.ts,.js}'],
  synchronize: false,
  logging: false,
  poolSize: 20, // Replicas typically handle more concurrent reads
});
```

Output:

```
// No output — construction only. Initialise both in server.ts.
```

4.2 Initialising Multiple DataSources in Parallel (Stage 2)

When you have multiple data sources that are independent of each other, initialise them in parallel with `Promise.all` rather than sequentially:

src/server.ts

```
import app from './app';
import { WriteDataSource, ReadDataSource } from './data-source';

const PORT = parseInt(process.env.PORT ?? '3000');

async function startServer(): Promise<void> {
  // Initialise both data sources concurrently
  await Promise.all([
    WriteDataSource.initialize(),
    ReadDataSource.initialize(),
  ]);
```

```
  console.log('[DB] WriteDataSource initialised.');
  console.log('[DB] ReadDataSource initialised.');

  app.listen(PORT, () => {
    console.log(`[Server] Listening on http://localhost:${PORT}`);
  });
}

startServer().catch((err) => {
  console.error('[Fatal]', err);
  process.exit(1);
});
```

Output:

```
[DB] WriteDataSource initialised.
[DB] ReadDataSource initialised.
[Server] Listening on http://localhost:3000
```

4.3 Selecting a DataSource Per Request (Stage 3)

In a service layer, use the appropriate source based on the operation type. A clean pattern is to create a thin abstraction that encapsulates the routing decision:

src/db.ts

```
import { DataSource, EntityManager } from 'typeorm';
import { WriteDataSource, ReadDataSource } from './data-source';

type Operation = 'read' | 'write';

export function getManager(op: Operation): EntityManager {
  const source: DataSource = op === 'write' ? WriteDataSource : ReadDataSource;

  if (!source.isInitialized) {
    throw new Error(`DataSource for '${op}' operations is not initialised.`);
  }

  return source.manager;
}
```

src/services/user.service.ts

```
import { getManager } from '../db';

// User entity is covered in Section 11 — referenced here
// as a placeholder to show the service pattern.
export async function countUsers(): Promise<number> {
  // Reads go to the replica
  const manager = getManager('read');
  // In Section 11 you will use manager.find(UserEntity) etc.
  // Here we use a raw query to avoid a forward reference:
  const result = await manager.query('SELECT COUNT(*) AS cnt FROM "user"');
  return parseInt(result[0].cnt);
}
```

Output:

```
// countUsers() would return e.g. 42 when the user table has 42 rows.
// Query executes against the ReadDataSource (replica).
```

Note — TypeORM Built-in Replication: TypeORM's PostgreSQL and MySQL drivers support a `replication` option at the `DataSource` level, which handles read/write routing automatically using a single `DataSource`. The explicit two-source pattern shown above gives you more control and clearer failure modes, which is preferable when you are learning the lifecycle before adding the abstraction.

---

5. Connection Pooling

A connection pool maintains a set of pre-opened database connections that are reused across requests. Opening a new TCP connection to a database takes time (often 20–100 ms); a pool eliminates that cost for the majority of queries.

5.1 How the Pool Works

1. On `initialize()`, TypeORM opens a minimum number of connections (`min`) immediately.

2. As queries arrive, the pool hands out available connections. When all are in use and more arrive, the pool opens additional connections up to `max`.
3. When a query finishes, its connection is returned to the pool — not closed.
4. Connections idle beyond a timeout are closed to free resources on the database server.

5.2 Configuring the Pool

Pool configuration is set via `poolSize` (for the default pool max) or the driver-native `extra` object for more granular control:

src/data-source.ts

```
import { DataSource } from 'typeorm';
import * as dotenv from 'dotenv';
dotenv.config();

export const AppDataSource = new DataSource({
  type: 'postgres',
  host:     process.env.DB_HOST     ?? 'localhost',
  port:     parseInt(process.env.DB_PORT ?? '5432'),
  username: process.env.DB_USER     ?? 'postgres',
  password: process.env.DB_PASSWORD ?? '',
  database: process.env.DB_NAME     ?? 'my_app_db',
  entities: [__dirname + '/entities/**/*{.ts,.js}'],
  synchronize: false,

  // Top-level pool max (equivalent to extra.max for pg/mysql2 drivers)
  poolSize: 10,

  // Driver-native pool options (pg / node-postgres)
  extra: {
    max:                10,   // Maximum pool size
    min:                2,    // Keep at least 2 connections open
    idleTimeoutMillis:  30000, // Close idle connections after 30 s
    connectionTimeoutMillis: 5000, // Throw if no connection available in 5 s
  },

  // Fail fast if the DB is unreachable on startup
  connectTimeoutMS: 5000,
});
```

Output:

```
// On initialize(), TypeORM will immediately open 'min' (2) connections.
// Up to 10 concurrent queries can run simultaneously.
// If all 10 are busy and an 11th query arrives, it will wait up to
// connectionTimeoutMillis (5000 ms) before throwing an error.
```

5.3 Pool Sizing Guidelines

Workload	Suggested Max Pool Size	Reasoning
Development / local testing	2–5	Avoid exhausting a shared local DB instance; surface connection errors early.
Low-traffic API (hobby / staging)	5–10	Headroom for burst traffic without over-provisioning.
Production API (moderate load)	10–20	Balance between throughput and DB server limits.
High-concurrency API	20–50+	Profile first. PostgreSQL's default `max_connections` is 100 — the sum of all client pools must not exceed this.

Important — Database Server Limits: Each pooled connection consumes a slot in the database server's `max_connections` setting. If you run four Node.js processes each with a pool of 25, that is 100 connections — hitting PostgreSQL's default limit and leaving none for administrative tooling. Set `poolSize` per-process, not per-application.

6. Testing Database Connectivity

Before deploying or during CI, it is valuable to assert that your `DataSource` configuration is valid and the database is reachable. There are two levels of connectivity testing: *connection-only* and *query-round-trip*.

6.1 Connection-Only Test

Calling `initialize()` and then `destroy()` is sufficient to confirm that the credentials and host are valid:

src/scripts/test-connection.ts

```
import { AppDataSource } from '../data-source';

async function testConnection(): Promise<void> {
  console.log('Testing database connection...');

  try {
    await AppDataSource.initialize();
    console.log('[OK] Connected to database:', AppDataSource.options.database);
    console.log('[OK] Driver type:',          AppDataSource.options.type);
    console.log('[OK] isInitialized:',        AppDataSource.isInitialized);
  } catch (err) {
    console.error('[FAIL] Could not connect:', (err as Error).message);
    process.exit(1);
  } finally {
    if (AppDataSource.isInitialized) {
      await AppDataSource.destroy();
      console.log('[OK] Connection closed cleanly.');
    }
  }
}

testConnection();
```

Run it with:

```
npx ts-node src/scripts/test-connection.ts
```

> Output:

```
Testing database connection...
[OK] Connected to database: my_app_db
[OK] Driver type:           postgres
[OK] isInitialized:         true
[OK] Connection closed cleanly.
```

6.2 Query Round-Trip Test

A connection test proves the TCP handshake succeeded. A round-trip test proves the database can also execute SQL — catching issues like missing permissions or invalid schemas:

src/scripts/test-connection.ts

```
import { AppDataSource } from '../data-source';

async function testConnection(): Promise<void> {
  console.log('Testing database connection...');

  try {
    await AppDataSource.initialize();
    console.log('[OK] Initialised.');

    // Execute a trivial SQL query to prove end-to-end connectivity
    const result = await AppDataSource.query('SELECT 1 AS ping');
    console.log('[OK] Query round-trip succeeded. Response:', result);

    // Confirm the target schema exists (PostgreSQL-specific)
    const schemaResult = await AppDataSource.query(
      `SELECT schema_name
       FROM information_schema.schemata
       WHERE schema_name = current_schema()`
    );
    console.log('[OK] Current schema:', schemaResult[0].schema_name);

  } catch (err) {
    console.error('[FAIL]', (err as Error).message);
    process.exit(1);
  } finally {
    if (AppDataSource.isInitialized) {
      await AppDataSource.destroy();
    }
  }
```

```
}

testConnection();
```

Output:

```
Testing database connection...
[OK] Initialised.
[OK] Query round-trip succeeded. Response: [ { ping: 1 } ]
[OK] Current schema: public
```

6.3 Health Check Endpoint in Express

Expose database status through a dedicated HTTP endpoint so that load balancers and monitoring tools can probe it:

src/routes/health.router.ts

```
import { Router, Request, Response } from 'express';
import { AppDataSource } from '../data-source';

const router = Router();

router.get('/health', async (_req: Request, res: Response): Promise<void> => {
  const dbOk = AppDataSource.isInitialized;
  let queryOk = false;

  if (dbOk) {
    try {
      await AppDataSource.query('SELECT 1');
      queryOk = true;
    } catch {
      queryOk = false;
    }
  }

  const status = dbOk && queryOk ? 'healthy' : 'degraded';
  const code   = status === 'healthy' ? 200 : 503;

  res.status(code).json({
    status,
    database: {
      initialised: dbOk,
```

```
      queryable:   queryOk,
    },
    timestamp: new Date().toISOString(),
  });
});

export default router;
```

Output:

```
GET /health (when DB is up):
{
  "status": "healthy",
  "database": {
    "initialised": true,
    "queryable": true
  },
  "timestamp": "2026-03-01T10:42:00.000Z"
}

GET /health (when DB is down):
{
  "status": "degraded",
  "database": {
    "initialised": false,
    "queryable": false
  },
  "timestamp": "2026-03-01T10:42:05.000Z"
}
```

Note: The health endpoint executes a live query on every call. For high-frequency polling (e.g., every 5 seconds from a load balancer), this is acceptable — `SELECT 1` is negligibly cheap. However, if you have concerns about query overhead, cache the result of the last health check in a module-level variable and refresh it on a timer instead.

---

7. Project Structure for This Section

The files introduced in this subtopic fit into this layout:

```
my-app/
├── src/
│   ├── data-source.ts           ← DataSource definition(s)
│   ├── app.ts                   ← Express app setup
│   ├── server.ts                ← Entry point: init DB, then listen
│   ├── db.ts                    ← Helper: getManager(op)
│   ├── entities/                ← Entity classes go here (Section 11)
│   ├── migrations/              ← Migration files go here (Section 12)
│   ├── routes/
│   │   └── health.router.ts     ← /health endpoint
│   └── scripts/
│       └── test-connection.ts   ← Standalone connectivity test
├── .env
├── package.json
└── tsconfig.json
```

Summary

Concept	Key Point
`DataSource` constructor	Synchronous; sets up metadata only — no network activity occurs until `initialize()`.
`initialize()`	Async; opens the pool and connects to the database. Must resolve before accepting HTTP requests.
`destroy()`	Async; closes all pool connections cleanly. Call during graceful shutdown inside `server.close()`.
`isInitialized`	Boolean property; `true` after a successful `initialize()`, `false` after `destroy()`.

Concept	Key Point
Entity glob patterns	Use `__dirname + '/entities/**/*{.ts,.js}'` to avoid having to list every entity class manually.
Multiple DataSources	Create separate named instances for read/write splitting or multi-tenancy; initialise them with `Promise.all()`.
Connection pooling	Controlled via `poolSize` and the driver-native `extra` object. Pool max × process count must not exceed the DB server's `max_connections`.
Connectivity testing	A standalone script validates credentials; a `SELECT 1` round-trip proves query execution. Expose both results via a `/health` endpoint.
`synchronize: false`	Never enable automatic schema sync in production. Use migrations (Section 12) for schema changes.

Interview Questions

1. What is the difference between constructing a `DataSource` and calling `initialize()` on it?
Expected answer: The constructor is synchronous and only reads configuration, builds entity metadata, and prepares internal structures — no network activity occurs. `initialize()` is asynchronous; it establishes the actual TCP connection(s) to the database and populates the connection pool. The two-step pattern allows you to configure the data source at module load time and defer the expensive network operation until the application is ready to start.

2. Why should `destroy()` be called inside the `server.close()` callback rather than before it?
Expected answer: `server.close()` stops accepting new connections but waits for in-flight requests to complete before firing its callback. If you call `DataSource.destroy()` before that callback, any

queries still running for those in-flight requests will fail because their pooled connection will have been closed. Destroying the pool inside the callback guarantees that all HTTP traffic has finished before database connections are torn down.

3. What are two valid reasons to run multiple `DataSource` instances in the same application?
Expected answer: Any two of: (1) Read/write splitting — direct write operations to a primary and read operations to a replica to reduce primary load; (2) Multi-tenancy — each tenant has a separate database and the application selects the appropriate data source per request; (3) Heterogeneous databases — different parts of the domain use different database engines (e.g., PostgreSQL for relational data, SQLite for testing or embedded use).

4. Your application runs as four identical Node.js processes on a server. Each process has a pool size of 30. The PostgreSQL instance has `max_connections = 100`. What is the problem and how would you fix it?
Expected answer: Four processes × 30 connections = 120 pooled connections, which exceeds the PostgreSQL limit of 100. The database will refuse connections beyond 100, causing requests to fail unpredictably. The fix is to reduce `poolSize` so that total_processes × poolSize stays well below `max_connections`, leaving some spare connections for administrative tools — e.g., four processes × 20 connections = 80, leaving 20 for administration.

5. What does `AppDataSource.isInitialized` tell you, and how would you use it in a health check?
Expected answer: `isInitialized` is a boolean that is `true` after a successful `initialize()` call and `false` after `destroy()` or before initialisation. In a health check endpoint, you first check `isInitialized` to short-circuit quickly if the data source is not running, and then execute a trivial query (`SELECT 1`) to prove end-to-end query capability. Return HTTP 200 if both pass, HTTP 503 if either fails.

6. Why should `synchronize: false` always be set in production, and what should be used instead?
Expected answer: `synchronize: true` causes TypeORM to automatically alter the database schema at application startup to match the current entity definitions. In production this is dangerous: it

can drop columns, alter types, or perform other destructive changes without any review or rollback capability. The correct alternative is to use TypeORM migrations, which are explicit, versioned SQL scripts that can be tested, reviewed, and rolled back in a controlled manner.

7. You want to initialise two independent `DataSource` instances at startup. Should you use `await ds1.initialize(); await ds2.initialize();` or `await Promise.all([ds1.initialize(), ds2.initialize()]);`? Why?
Expected answer: Use `Promise.all()`. The sequential `await` form means that `ds2.initialize()` does not start until `ds1.initialize()` has fully resolved, adding latency equal to the slower of the two connections. Since the two data sources are independent, they can be initialised concurrently. `Promise.all()` fires both calls simultaneously and resolves when both complete, minimising total startup time. If either fails, the promise rejects and the startup can be aborted.

8. What is `connectionTimeoutMillis` in the pool's `extra` options, and when would it apply?
Expected answer: `connectionTimeoutMillis` defines how long (in milliseconds) a query will wait for a connection to become available from the pool before throwing an error. It applies when all connections in the pool are in use and the pool has reached its maximum size — so no new connections can be opened. Without this timeout, a query would wait indefinitely. Setting an explicit timeout causes it to fail fast with a clear error, which is preferable to silent queue build-up under heavy load.

Practical Assignment — Multi-DataSource Server with Health Check

Objective: Build a small Express application that initialises two `DataSource` instances (a primary and a read replica — both pointing at the same local database for this exercise), exposes a `/health` endpoint reporting on both, and shuts down gracefully on `SIGINT`.

Prerequisites: PostgreSQL running locally (or a Docker container). Node.js with TypeScript configured (see Section 10-02).

1. Create the project skeleton and install dependencies:
   ```
   mkdir typeorm-datasource-lab && cd typeorm-datasource-lab
   npm init -y
   npm install typeorm pg express dotenv reflect-metadata
   npm install --save-dev typescript ts-node @types/express @types/node
   npx tsc --init
   ```
2. Create a `.env` file with your database credentials. For this exercise, both the primary and replica point to the same local database (change host/port if you have a real replica): `.env`
   ```
   DB_PRIMARY_HOST=localhost
   DB_PRIMARY_PORT=5432
   DB_REPLICA_HOST=localhost
   DB_REPLICA_PORT=5432
   DB_USER=postgres
   DB_PASSWORD=secret
   DB_NAME=lab_db
   PORT=3000
   ```
3. Create `src/data-source.ts` with two named `DataSource` instances (`WriteDataSource` and `ReadDataSource`) as shown in Section 4.1 of these notes. Set `poolSize: 3` on each.
4. Create `src/routes/health.router.ts` that checks *both* data sources. The response body should be:
   ```
   {
     "status": "healthy" | "degraded",
     "sources": {
       "write": { "initialised": true, "queryable": true },
       "read":  { "initialised": true, "queryable": true }
     },
     "timestamp": "..."
   }
   ```
5. Create `src/app.ts` that sets up Express and mounts the health router.
6. Create `src/server.ts` that initialises both data sources with `Promise.all()`, starts the server, and registers `SIGINT` / `SIGTERM` handlers that destroy both data sources before exiting.

7. Add a `start` script to `package.json`:
   ```
   "start": "ts-node src/server.ts"
   ```
8. Run the server and verify the health endpoint:
   ```
   npm start
   ```
   In a separate terminal: `curl http://localhost:3000/health`
9. Press `Ctrl+C` and confirm both data sources report clean shutdown in the console.

Expected Output (curl):

Output:

```
{
  "status": "healthy",
  "sources": {
    "write": { "initialised": true, "queryable": true },
    "read":  { "initialised": true, "queryable": true }
  },
  "timestamp": "2026-03-01T10:42:00.000Z"
}
```

Expected Shutdown Output (terminal):

Output:

```
[Server] SIGINT received — shutting down.
[Server] HTTP server closed.
[DB] WriteDataSource destroyed.
[DB] ReadDataSource destroyed.
```

Bonus Challenge: Add a `/health/pool` endpoint that reads the actual pool statistics from each data source. The `pg` driver exposes pool internals via `(AppDataSource.driver as any).master` (a `pg.Pool` instance with `totalCount`, `idleCount`, and `waitingCount` properties). Return these counts in the JSON response. This is the kind of endpoint that makes a real difference in production observability.

Additions & Changes from Syllabus Scope (reviewer note):

The syllabus lists "Working with DataSource: Initialization, options", "Multiple Data Sources and API: Switching connections", and "Connection Pooling and Testing" as the three bullet points for this subtopic. Coverage is complete and in that order. The following additions were made that go slightly beyond the literal bullet text but serve the subtopic's learning goals: (1) A dedicated *Graceful Shutdown* stage was added to the initialisation section — this is the natural completion of the initialisation lifecycle and students are very likely to encounter it immediately in practice. (2) A *Health Check Endpoint* was included under connectivity testing, as this is the canonical Express pattern for surfacing DataSource status and was a logical extension of the "testing" bullet. (3) A project folder-tree diagram was added to give students a clear mental model of where each file lives before they encounter entities and migrations in Sections 11–12. No content from future sections (entities, migrations, QueryBuilder) is assumed or used — forward references are limited to brief placeholders with explicit acknowledgements.